안녕하세요.
Javascript 강좌중인 Falcon 입니다.

생성자와 new라는 키워드에 대해서 배워볼텐데 이 부분은 조금 많이 어렵습니다. 제가 여러분들에게 객체를 만드는 방법을 알려드린적이 있습니다. 한번 복습을 해보도록 하죠.

```
<script type="text/javascript">
  var person = {
    'nick' : 'Falcon',
    'showData' : function() {
      return 'My nick is ' + person.nick;
    }
  }
  console.log(person.showData());
</script>
```

이미 배웠던 내용이기 때문에 따로 설명을 해드리지는 않겠습니다. 그런데 객체를 생성할 수 있는 다든 방법이 있습니다. 위와 같지만 생성하는 방법이 조금 다를 뿐 입니다.

```
<script type="text/javascript">
  var person = {};
  person.nick = "Falcon";
  person.showData = function() {
    return 'My nick is ' + person.nick;
  }

  console.log(person.showData());
</script>
```

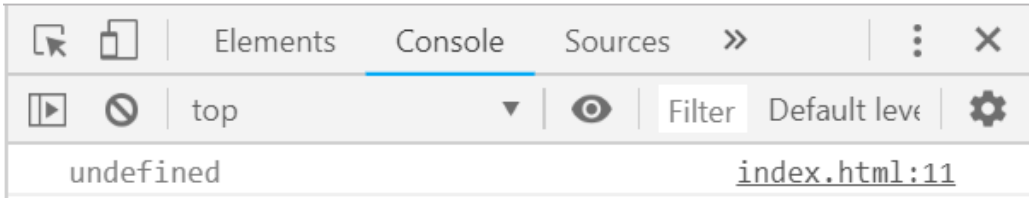
이 방법은 객체의 접근 방법을 이용한 객체 생성 방법입니다. 두가지 방법 중에 하나를 선택해서 사용하시면 되는데 아마 상황에 따라서 쓰게 될 겁니다. 아래의 코드들을 보시면 그 이유를 알게 될 것 입니다. 그렇다면 지금부터 객체를 하나 더 생성한다고 가정을 해보면서 강좌를 잘 읽어주시길 바랍니다.

```
8        <script type="text/javascript">
9          var person1 = {};
10         person.nick = "Falcon";
11         person.showData = function() {
12           return 'My nick is ' + person.nick;
13         }
14
15         var person2 = {};
16         person.nick = "spider";
17         person.showData = function() {
18           return 'My nick is ' + person.nick;
19         }
20
21         console.log(person1.showData());
22         console.log(person2.showData());
23       </script>
```

우리가 지금까지 배운 방법대로라면 객체를 하나 더 생성해서 이런식으로 만들어주면 될 것 입니다. 그런데 이는 비효율적인 방법입니다. 이러한 방식으로 추가를 하다보면 유지보수에도 문제가 생기게 됩니다.

```
8        <script type="text/javascript">
9          function person() {};
10         var p = person();
11         console.log(p);
12       </script>
```

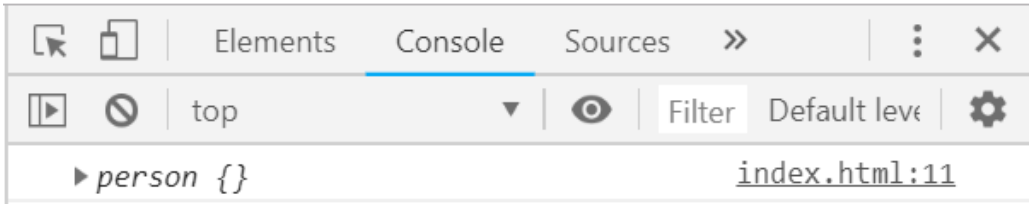
제가 함수는 객체다라고 설명을 해드렸는데 그 이유는 제가 강좌로 쓴 C++과 자바라는 언어에는 클래스라는 것이 존재합니다. 그리고 그 클래스 내에는 생성자라는 것이 존재하는데 Javascript의 생성자는 함수를 이용합니다. 위의 코드의 경우는 일반적인 함수를 나타냅니다. 그걸 보여드리기 위해서 var p에 함수를 넣었습니다.



그리고 위의 코드를 출력해보면 undefined가 뜨게 됩니다. 이유는 함수에 아무런 값이 입력되지 않았기 때문입니다. 그런데 new라는 키워드가 앞에 붙는다면 결과는 달라지게 됩니다.

```
8   <script type="text/javascript">
9     function person() {};
10    var p = new person();
11    console.log(p);
12  </script>
```

10행 부분을 보면 person() 앞에 new라는 키워드가 붙었습니다. 그리고나서 다시 한번 이를 출력해보면



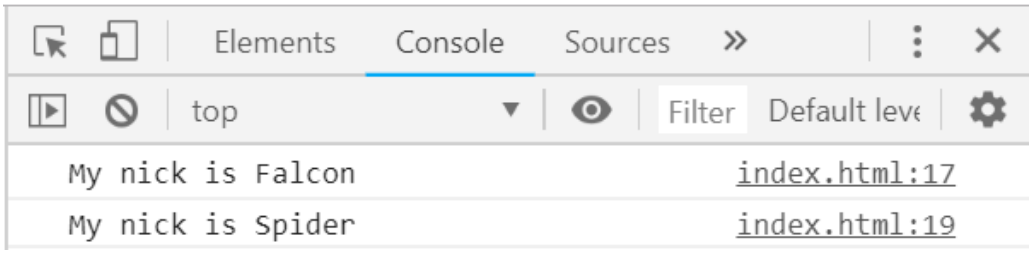
undefined로 뜨던 결과값이 person 객체의 빈 객체로 나오게 됩니다. 즉, new라는 키워드를 통해서 함수가 객체로 변해버렸습니다. 그래서 함수도 하나의 객체다라는 표현을 쓴 것 입니다.

```
8   <script type="text/javascript">
9     function person() {};
10    var p1 = new person();
11    p1.nick = 'Falcon';
12    p1.showData = function() {
13      return 'My nick is ' + p1.nick;
14    }
15
16    var p2 = new person();
17    p2.nick = 'spider';
18    p2.showData = function() {
19      return 'My nick is ' + p2.nick;
20    }
21
22    console.log(p1.showData());
23    console.log(p2.showData());
24  </script>
```

위에서 보여드린 코드와 아래의 코드를 비교해보시기 바랍니다. 여기서 중요한 것은 new라는 키워드를 통해서 함수가 객체가 되었다는 것, 그리고 p1과 p2는 서로 다른 객체라는 것 입니다. 그 부분을 코드를 통해서 보여드리고 있습니다. 하지만 코드의 중복이 일어나는 것은 위에서 보여드린 방법과 별로 달라지지 않았다는 것 입니다.

```
8   <script type="text/javascript">
9     function person(nick) {
10      this.nick = nick;
11      this.showData = function() {
12        return 'My nick is ' + this.nick;
13      }
14    };
15
16    var p1 = new person('Falcon');
17    console.log(p1.showData());
18    var p2 = new person('Spider');
19    console.log(p2.showData());
20  </script>
```

9행에서 보이는 함수 옆에 있는 부분이 매개변수였지만 new라는 키워드가 붙음으로써 이 부분은 매개변수가 아닌 생성자라고 생각하셔야 합니다. 여기서 중요한 것은 new라는 키워드가 있고 없고의 차이입니다. 그리고 여기서도 this라는 것이 보이는데 여기서의 this는 16행과 18행에서 보이는 p1과 p2의 안에 들어있는 person 객체 입니다. 이 둘은 엄연히 다른 객체를 들고 있습니다. 안의 내용물만 같습니다. 이 부분에서 중요한 것은 person안에 든 내용에 대한 중복이 사라지면서 객체 하나로 100개 1000개에 대한 것을 만들 수 있다는 점입니다.



하나의 객체를 가지고 두개를 출력하는 것을 결과 값을 통해 알 수 있습니다. 나름대로 정리를 해보았는데 여러분들에게는 혼란이 오지 않을까 조금 두렵기도 하네요. 그럼 저는 이만~!

이것으로 [Javascript 강좌] 제 21강 생성자와 new 편을
마치도록 하겠습니다.